

PVT

MT8676 Hypervisor Suspend & Resume

sample

版本: v1
出版日期: 2026/1/10

版本记录

版本	日期	作者	描述
v1	2026/1/10	杨娟陶	正式版

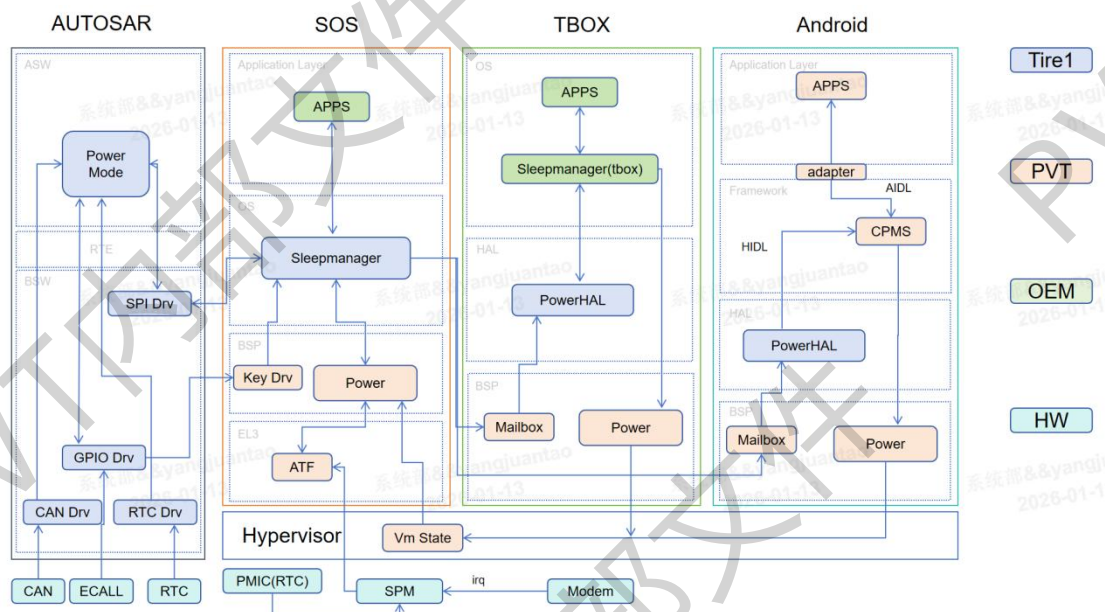
目录

1. 参考案例主要架构介绍	4
1.1 参考方案总架构介绍	4
1.2 休眠流程介绍	4
1.3 唤醒流程介绍	5
1.3.1 唤醒场景示范	5
1.3.2 读取唤醒源	5
1.3.2.1 MCU 唤醒源	6
1.3.2.2 SOC 唤醒源	6
1.3.3 场景设计示范	8
1.3.3.1 CAN 网络唤醒流程	7
1.3.3.2 长连接唤醒流程	8
1.3.3.3 Modem 唤醒流程	9
1.3.3.4 modem 唤醒后的远控场景	11
1.3.3.5 预约功能唤醒流程	12
2. MCU 扩展	14
2.1 MCU/SOC 沟通 gpio 设定	14
2.2 使用场景介绍	14
2.2.1 CAN 唤醒	14
2.2.2 休眠	14
2.2.3 SOC 唤醒 MCU	15
2.3 MCU 保活机制	15

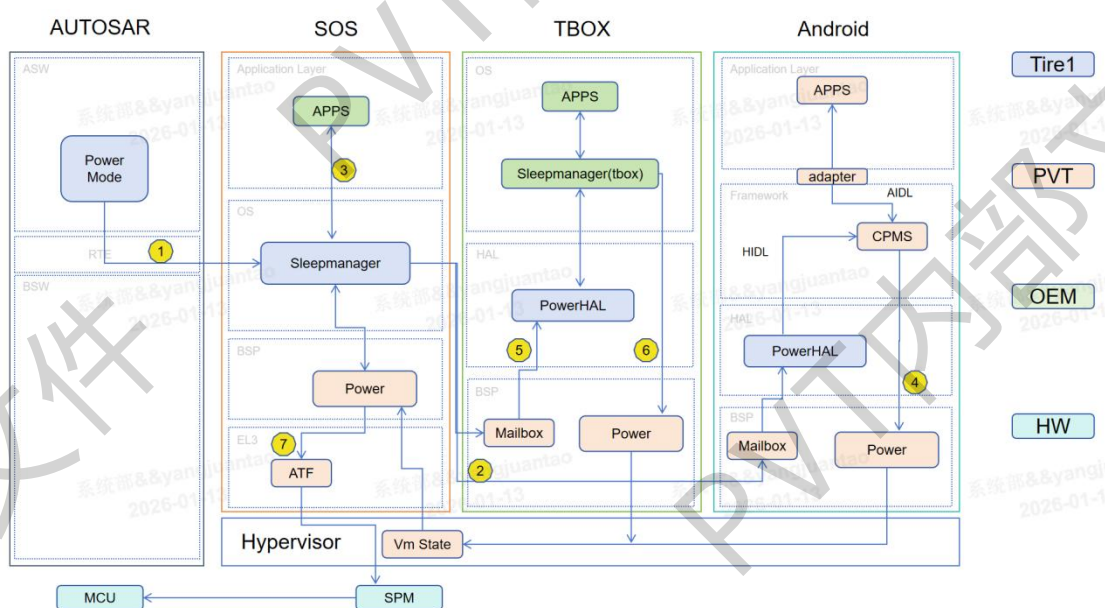
1. 参考案例主要架构介绍

1.1 参考方案总架构介绍

此为当前可扩展的一种架构介绍。主要是触发场景的完善和多系统电源管理总控(sleepmanager)的改变。



1.2 休眠流程介绍



- 1)、PowerMode 决策为休眠模式后，变更电源模式并通知，再拉高 SOC 休眠 gpio 通知 SOC 休眠。sleepmanager 收到虚拟的 powerkey 休眠后，触发 SOC 的休眠流程。
- 2)、SOS 通过 mailbox 通知 android 开始休眠。

- 3)、SleepManager 通知仪表各注册 APP 执行休眠流程。
- 4)、SleepManager 通过虚拟机读取收到 Android 休眠完成消息。
- 5)、SleepManager 通过 mailbox 通知触发 TBOX 的休眠。Sleepmanager(tbox)收到 powerkey 后, 根据需求触发云端上报, 上报完成后开始触发 TBOX 休眠流程。
- 6)、SleepManager 通过虚拟机读取收到 TBOX 也休眠完成。
- 7)、SOC ATF 拉低 SOC 状态 gpio 以通知 MCU 当前 SOC 状态为休眠。MCU 收到后开始自己的休眠流程。

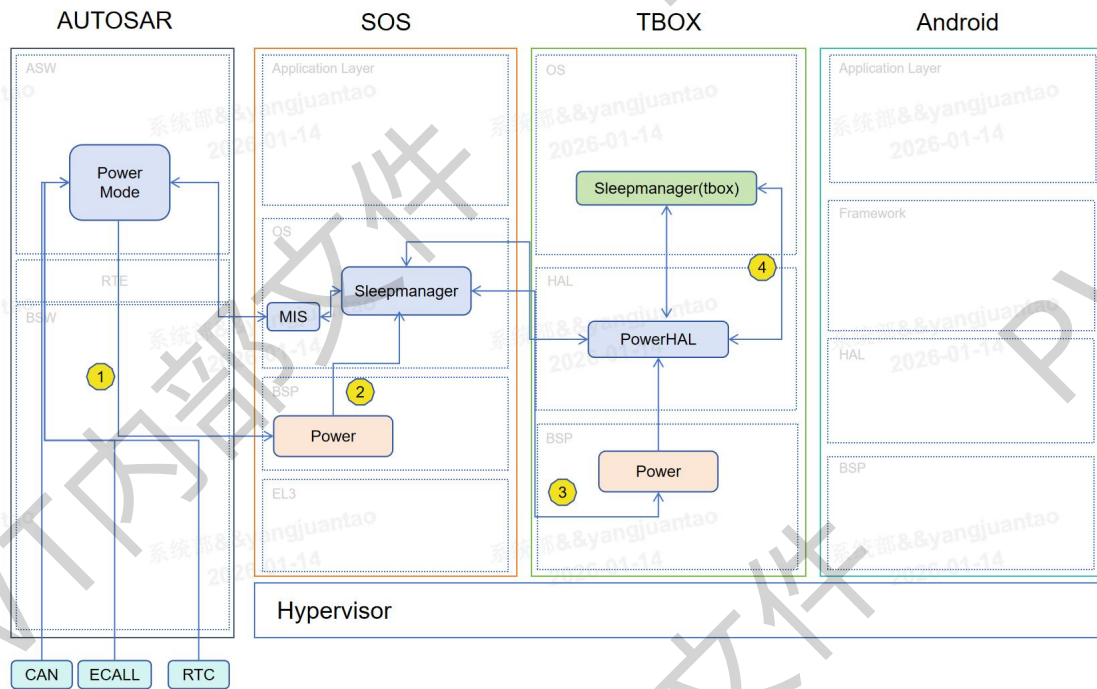
1.3 唤醒流程介绍

1.3.1 唤醒场景示范

唤醒业务	唤醒源	首唤醒系统	识别模块	适用深休眠	适用浅睡眠	唤醒所属业务	mcu运行	sos运行	tbox运行	android运行	直接拉起android
can网络恢复	CAN	MCU	PowerMode	√	√	can唤醒HUT	√	√	√	√	√
7天计时	RTC-MCU	MCU	PowerMode	×	√	唤醒TBOX上传后深休眠	√	√	√	×	×
IGN硬线	硬线IO	MCU	PowerMode	√	√	KL15唤醒HUT	√	√	√	√	√
ECALL物理按键	硬线IO	MCU	PowerMode	×	√	触发ECALL业务	√	√	√	×	×
短信唤醒	Modem-sms	SOS	SleepManager	×	√	处理短信唤醒的远控和非远控	√	√	√	×	×
数字远控	Modem-data	SOS	SleepManager	×	√	处理数据业务的唤醒进行远控	√	√	√	×	×
ECALL回拨	Modem-call	SOS	SleepManager	×	√	ECALL回拨唤醒tbox	√	√	√	×	×
长连接	RTC-SOC	SOS	SleepManager	×	√	唤醒tbox与TSP心跳同步	√	√	√	×	×
预约升级	RTC-SOC	SOS	SleepManager	×	√	唤醒tbox开启升级	√	√	√	√	×
预约充电	RTC-SOC	SOS	SleepManager	×	√	唤醒tbox开启充电	√	√	√	√	×
智能闹钟	RTC-SOC	SOS	SleepManager	×	√	唤醒tbox触发一些	√	√	√	√	×
自动闭锁	RTC-SOC	SOS	SleepManager	×	√	解锁后15分钟唤醒起	√	√	√	√	×

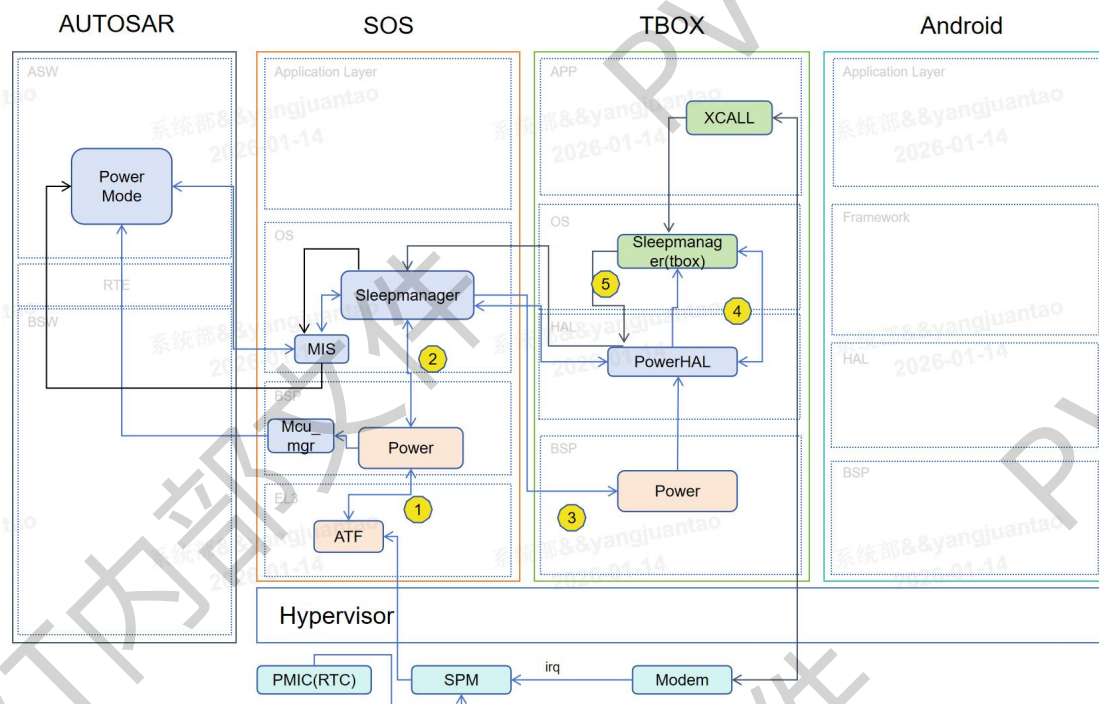
1.3.2 读取唤醒源

1.3.2.1 MCU 唤醒源



- 1)、MCU 通过 GPIO 唤醒 SOC, 用 fdbus+spi 把唤醒源传给 soc 并写入/sys/power/wakeup_type 节点。
- 2)、SOS 的 SleepManager 模块唤醒后, 从/sys/power/wakeup_type 节点读出唤醒源。
- 3)、Sleepmanager 模块根据具体的唤醒原因决策需要唤醒 TBOX 系统后, 唤醒 TBOX 系统。
- 4)、TBOX 系统的 Sleepmanager(tbox)模块向 SOS 系统的 SleepManager 模块询问当前唤醒源 (CAN/IGN/ECALL/RTC) , 并执行对应的业务逻辑。

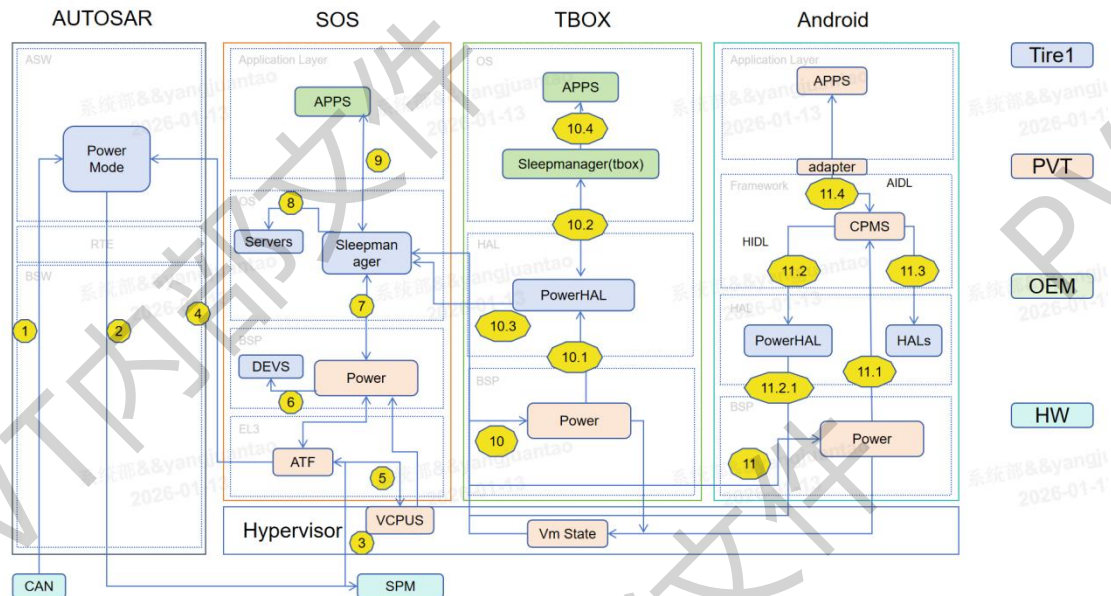
1.3.2.2 SOC 唤醒源



- 1)、SOC 上 RTC Modem 唤醒 SOS 系统，SOS 系统唤醒 MCU 系统。
- 2)、SOS 系统唤醒后，将唤醒源写入 `/sys/power/wakeup_type` 节点。
- 3)、SleepManager 模块根据具体的唤醒原因决策需要唤醒 TBOX 系统后，唤醒 TBOX 系统。
- 4)、TBOX 系统的 SleepManager(tbox)模块向 SOS 系统的 SleepManager 模块询问当前唤醒源 (Modem/RTC)，并执行对应的业务逻辑。
- 5)、TBOX 系统的 XCALL 模块根据 modem 中的信息将子因 (sms/call/data/modem fail) 给到 SleepManager(tbox)模块；SleepManager(tbox)模块通过 power hal 将子因转发给 SOS 系统的 SleepManager 和 MCU 的 PowerMode 模块。

1.3.3 场景设计示范

1.3.3.1 CAN 网络唤醒流程(IGN 硬线唤醒也适用)



- 1)、CAN 收发器收到 CAN 信号唤醒 MCU 系统
- 2)、MCU 系统的 PowerMode 模块拉高 gpio, 以通知 SOC 开始唤醒流程。
- 3)、SOC 上的 MTK SPM 模组收到 gpio EINT, 依次打开释放的资源, 如通知 PMIC 离开 Low Power Mode, 打开 VCORE 和 26M, 发送命令通知 DRAM 离开自刷新, 最后给 cpu0 上电。
- 4)、SOC 上的 ATF 模块在 cpu0 上电后, 执行 cpu 的唤醒流程, 再跳到 Suspend 前设定的地址, 开始执行系统 Resume 流程, 完成后拉 gpio, 以通知 MCU 当前 SOC 的运行状态为唤醒。
- 5)、系统 Resume 流程会唤醒虚拟机 vcpu 线程, 跳到 Host OS Linux Kernel 执行 kernel 的 power 唤醒流程。
- 6)、kernel power 模块经过 resume systemcore ->恢复全局 cpu 中断 ->拉起 nonboot cpus ->唤醒 dev(resume_noirq -> resume_early -> resume -> complete) ->取消冻结
- 7)、kernel power 模块执行唤醒完成后返回 SleepManager 模块收到 kernel power 的调用里。
- 8)、SleepManager 模块收到调用休眠返回后, 判定唤醒成功且唤醒源为 mcu 且为 can 网络, 唤醒中间件各业务服务。
- 9)、SleepManager 模块继续唤醒应用层各注册的客户端。
- 10)、SleepManager 模块通过写虚拟机节点唤醒 TBOX。
- 10.1)、TBOX 系统的 kernel power 经过 resume systemcore ->恢复全局 cpu 中断 ->拉起 nonboot cpus ->唤醒 dev(resume_noirq -> resume_early -> resume -> complete) ->取消冻结。然后返回 power hal 的调用函数。
- 10.2)、TBOX 系统的 power hal 模块收到返回唤醒后同步反馈给 SleepManager(tbox)。
- 10.3)、TBOX 系统的 power hal 模块同步开始唤醒消息给给 SleepManager。
- 10.4)、SleepManager(tbox)收到唤醒后, 判定唤醒成功且唤醒源为 can 网络, 唤醒中间件各业务服务。
- 11)、SleepManager 模块通过写虚拟机节点唤醒 android。
- 11.1)、Android 系统的 kernel power 经过 resume systemcore ->恢复全局 cpu 中断 ->拉起 nonboot

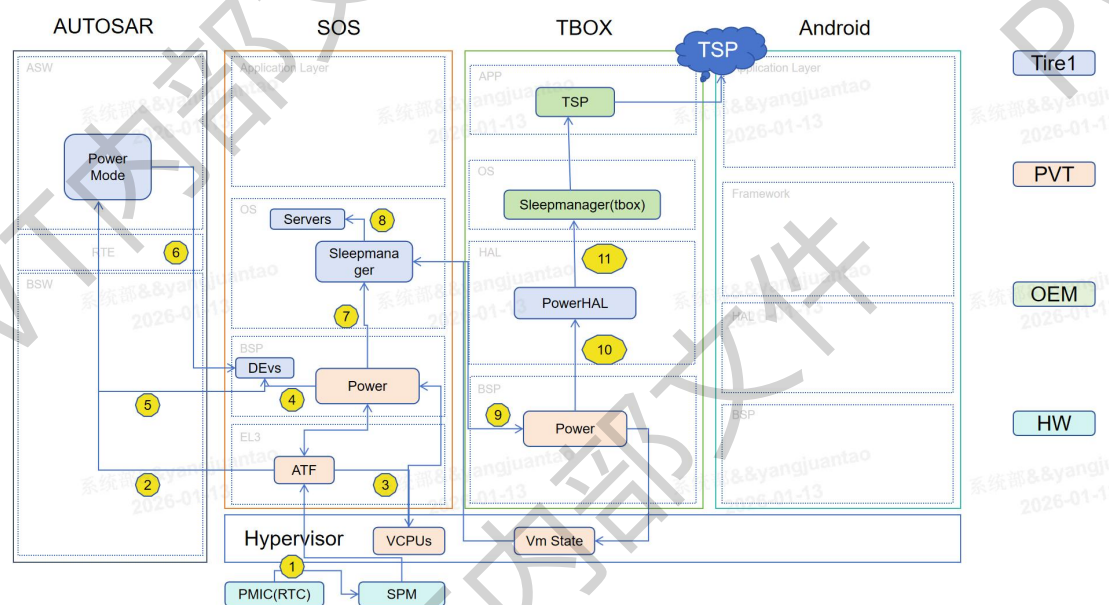
cpus ->唤醒 dev(resume_noirq->resume_early ->resume ->complete)->取消冻结。然后返回 CPMS 的休眠调用函数。

11.2)、CPMS 模块同步开始唤醒消息给给 power hal, power hal 同步开始唤醒消息给给 sleepmanager。

11.3)、CPMS 收到唤醒后, 判定唤醒成功, 通知各注册的 hal 进行唤醒。并同时恢复 bt wifi usb。

11.4)、CPMS 唤醒各业务服务。

1.3.3.2 长连接唤醒流程



1)、SOC上 PMIC 自带的 RTC 闹钟触发了 PMIC 离开 Low Power Mode, 打开 VCORE 和 26M, 发送命令通知 DRAM 离开自刷新, 最后给 cpu0 上电。

2)、SOC 上的 ATF 模块在 cpu0 上电后, 执行 cpu 的唤醒流程, 再跳到 Suspend 前设定的地址, 开始执行系统 Resume 流程。完成后拉 gpio, 以通知 MCU 当前 SOC 的运行状态为唤醒; 拉 gpio 以通知 MCU 唤醒。

3)、系统 Resume 流程会唤醒虚拟机 vcpu 线程, 跳到 Host OS Linux Kernel 执行 kernel 的 power 唤醒流程。

4)、kernel power 模块经过 resume_systemcore ->恢复全局 cpu 中断 ->拉起 nonboot cpus ->唤醒 dev(resume_noirq ->resume_early)

5)、mcu_mgr 模块在 resume_noirq 层被唤醒后判定当前是 SOC 自唤醒, ATF 中拉 gpio 让 MCU 唤醒, 阻塞并等待 MCU 的唤醒结果

6)、MCU 唤醒完成后拉 gpio 通知 SOC 自己唤醒完成。SOC 的 mcu_mgr 模块在收到 MCU 的唤醒结果后: 成功则继续唤醒;

7)、kernel power 模块检测到 resume_noirq 成功执行完成, 则继续唤醒 dev(resume_early->resume ->complete)->取消冻结, 然后唤醒 SleepManager 调用的休眠函数。

8)、SleepManager 模块收到调用休眠返回后, 判定唤醒成功且唤醒源为 soc rtc。

9)、SleepManager 模块通过写虚拟机节点唤醒 TBOX。

10)、TBOX 系统的 kernel power 经过 resume systemcore ->恢复全局 cpu 中断 ->拉起 nonboot cpus ->唤醒 dev(resume_noirq ->resume_early->resume ->complete)→取消冻结。然后返回 power hal 的调用函数。

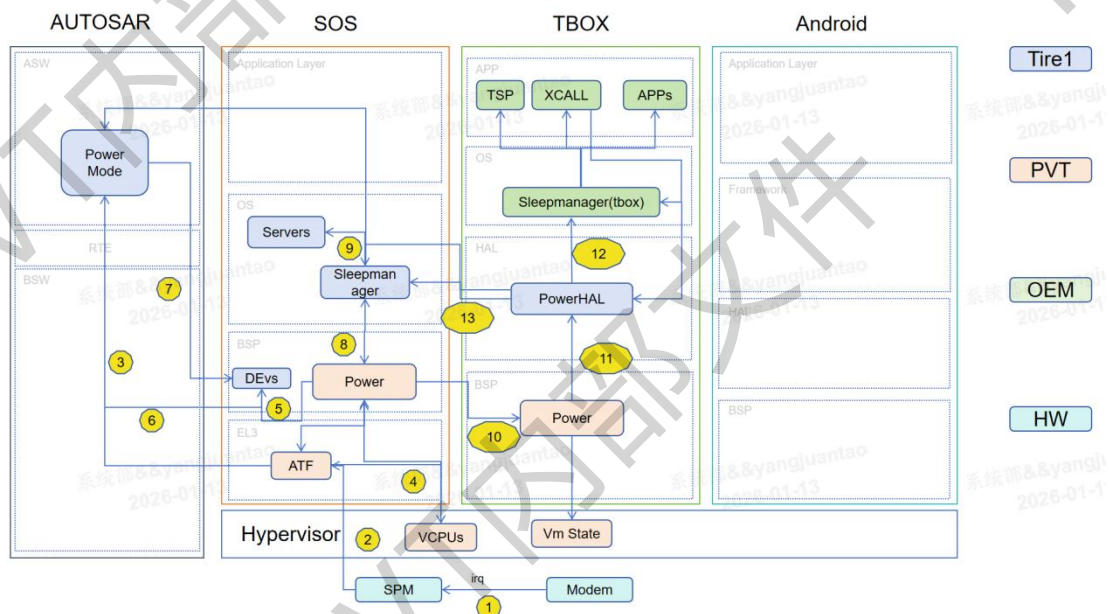
11)、TBOX 系统的 power hal 模块收到返回唤醒后同步反馈给 SleepManager(tbox)。

-----SleepManager(tbox)收到唤醒后, 判定唤醒成功且唤醒源为 SOC RTC 和闹钟事件后, 则唤醒中间件各业务服务, 并通知 TSP 模块。

-----TSP 模块收到闹钟事件通知后, 向 TSP 平台上报。

12)、TBOX 系统的 power hal 模块唤醒 SleepManager(tbox)后发送“开始唤醒消息”给 sleepmanager。

1.3.3.3 modem 唤醒流程（适用短信、ECALL、数据唤醒）



1)、SOC 上 Modem 接收到数据/短信/CALL 后通知 SPM。

2)、SOC 上的 MTK SPM 模组收到 modem 通知, 依次打开释放的资源, 如通知 PMIC 离开 Low Power Mode, 打开 VCORE 和 26M, 发送命令通知 DRAM 离开自刷新, 最后给 cpu0 上电。

3)、SOC 上的 ATF 模块在 cpu0 上电后, 执行 cpu 的唤醒流程, 再跳到 Suspend 前设定的地址, 开始执行系统 Resume 流程。完成后拉 gpio, 以通知 MCU 当前 SOC 的运行状态为唤醒; 拉 gpio 以通知 MCU 唤醒。

4)、系统 Resume 流程会唤醒虚拟机 vcpu 线程, 跳到 Host OS Linux Kernel 执行 kernel 的 power 唤醒流程。

5)、kernel power 模块经过 resume systemcore ->恢复全局 cpu 中断 ->拉起 nonboot cpus ->唤醒 dev(resume_noirq ->resume_early)

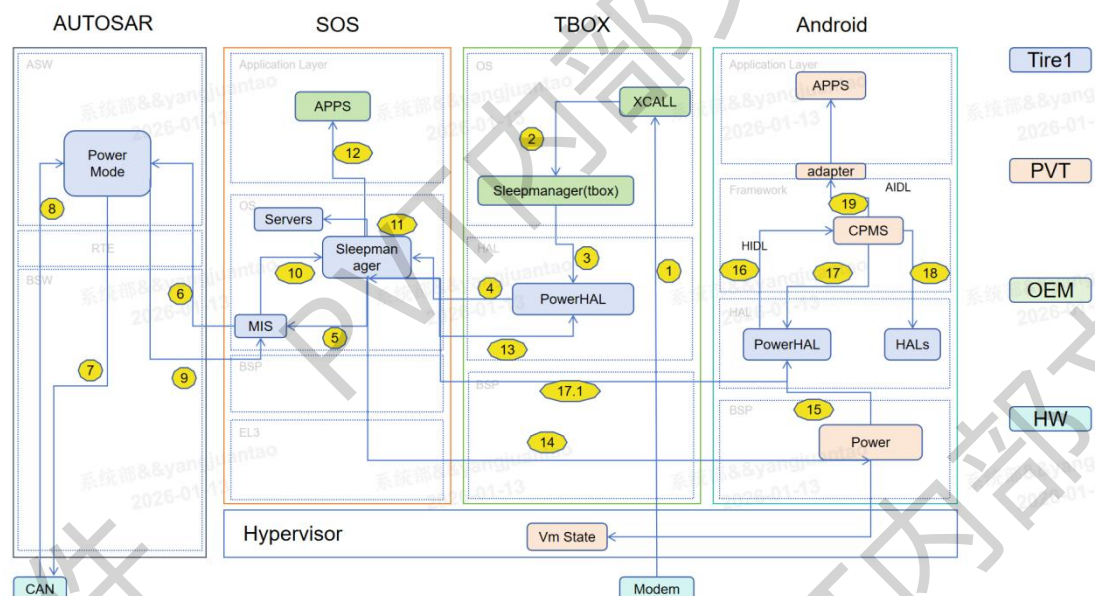
6)、mcu_mgr 模块在 resume_noirq 层被唤醒后判定当前是 SOC 自唤醒, 在 ATF 拉 gpio 让 MCU 唤醒, 阻塞并等待 MCU 的唤醒结果

7)、MCU 唤醒完成后拉 gpio 通知 SOC 自己唤醒完成。SOC 的 mcu_mgr 模块在收到 MCU 的唤醒结果后: 成功则继续唤醒;

8)、kernel power 模块检测到 resume_noirq 成功执行完成, 则继续唤醒 dev(resume_early->resume ->complete)→取消冻结, 然后唤醒 SleepManager 调用的休眠函数。

- 9)、SleepManager 模块收到调用休眠返回后, 判定唤醒成功且唤醒源为 soc modem。
- 10)、SleepManager 模块通过写虚拟机节点唤醒 TBOX。
- 11)、TBOX 系统的 kernel power 经过 resume systemcore -> 恢复全局 cpu 中断 -> 拉起 nonboot cpus -> 唤醒 dev(resume_noirq -> resume_early -> resume -> complete) -> 取消冻结。然后返回 power hal 的调用函数。
- 12)、TBOX 系统的 power hal 模块收到返回唤醒后同步反馈给 SleepManager(tbox)。
- SleepManager(tbox)收到唤醒后, 判定唤醒成功且唤醒源为 SOC modem 后, 则唤醒中间件各业务服务, 并通知 XCALL 模块进行 modem 读取和校验。
- XCALL 模块读取 modem 中数据/SMS/CALL 进行校验, 结果通知给 SleepManager(tbox)模块。
- SleepManager(tbox)模块通知子唤醒原因给 power hal 模块。
- power hal 模块通知子唤醒原因给 SOS 系统的 SleepManager 模块。
- SleepManager 模块根据结果通知 MCU 系统 PowerMode 模块当前状态。如果校验失败, MCU 系统 PowerMode 模块则发起休眠流程 STR; 如果校验成功, 则开启强制保活的倒计时 MCU_KEEP_TBOX_TIMER_OUT。
- 13)、TBOX 系统的 power hal 模块唤醒 SleepManager(tbox)后发送“开始唤醒消息”给 SleepManager。

1.3.3.4 modem 唤醒后的远控场景（响应远控信号的部分适用于预约功能）



- 1)、TBOX 系统上的 XCALL 读取 modem 中的数据/短信, 进行校验和识别。
- 2)、XCALL 模块识别到远控请求时, 发送 can 信号, 再通知 SleepManager(tbox)模块需要维持网络。
- 3)、SleepManager(tbox)模块将维持网络的请求发送给 power hal 模块。
- 4)、power hal 模块通过 fdbus(vsock)通知维持网络给 SOS 系统的 sleepmanager 模块。
- 5)、sleepmanager 模块记录维持网络的 log, 再将维持网络状态传递给 mcu_ipc_server 模块。
- 6)、mcu_ipc_server 模块将维持网络请求通过 SPI 传递给 MCU 系统的 PowerMode 模块。
- 7)、MCU 系统的 PowerMode 模块通过 CAN 网络持续强制发送网络管理报文给 CEM, 并根据消息中携带的参数重置对应时间的倒计时 MCU_KEEP_TBOX_TIMER_OUT, 倒计时结束则不再强制发送网络管

理报文，届时根据实际情况进行决策电源模式等。（只要有此消息则重置倒计时 MCU_KEEP_TBOX_TIMER_OUT）

8)、MCU 系统的 PowerMode 模块收到 CEM 发来的远程模式信号，变更电源模式为远控模式，不打破强制网络管理报文的计时。

9)、PowerMode 模块通过 SPI 通信通知 SOS 系统的 mcu_ipc_server 模块需要 can 唤醒。

10)、SOS 系统的 mcu_ipc_server 模块收到消息后，透传通知组 sleepmanager 模块。

11)、sleepmanager 模块收到需要 can 唤醒消息后，唤醒中间件各业务服务。

12)、sleepmanager 模块继续唤醒应用层各注册的客户端。

13)、sleepmanager 模块通知 can 唤醒给 TBOX 系统的 power hal 模块。

14)、sleepmanager 模块通过写虚拟机节点唤醒 android。

15)、Android 系统的 kernel power 经过 resume systemcore ->恢复全局 cpu 中断 ->拉起 nonboot cpus ->唤醒 dev(resume_noirq -> resume_early -> resume -> complete) ->取消冻结。然后返回 power hal 的调用函数。

16)、Android 系统的 power hal 模块收到返回唤醒后同步反馈给 CPMS。

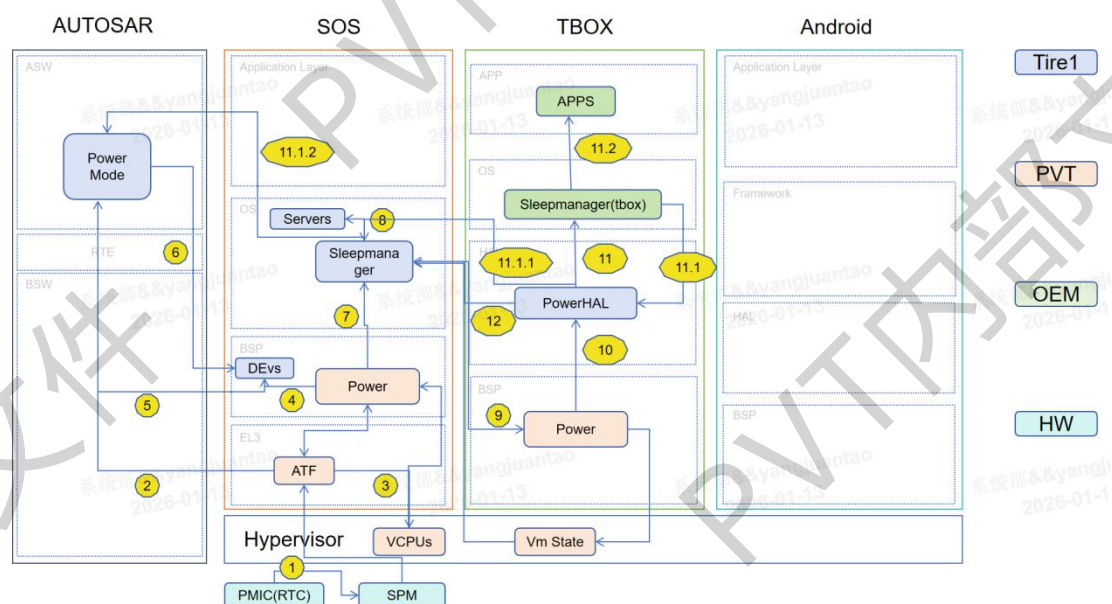
17)、CPMS 模块同步开始唤醒消息给 power hal 模块。

17.1)、power hal 模块给 SOS 系统的 sleepmanager 模块。sleepmanager 模块收到后进行 log 记录。

18)、CPMS 收到唤醒后，判定唤醒成功，通知各注册的 hal 进行唤醒。并同时恢复 bt wifi usb。

19)、CPMS 再唤醒各业务服务。

1.3.3.5 预约功能唤醒流程（适用自动闭锁、预约升级、智能闹钟、预约充电）



1)、SOC 上 PMIC 自带的 RTC 闹钟触发了 PMIC 离开 Low Power Mode，打开 VCORE 和 26M，发送命令通知 DRAM 离开自刷新，最后给 cpu0 上电。

- 2)、SOC 上的 ATF 模块在 cpu0 上电后, 执行 cpu 的唤醒流程, 再跳到 Suspend 前设定的地址, 开始执行系统 Resume 流程。完成后拉 gpio, 以通知 MCU 当前 SOC 的运行状态为唤醒。
- 3)、系统 Resume 流程会唤醒虚拟机 vcpu 线程, 跳到 Host OS Linux Kernel 执行 kernel 的 power 唤醒流程。
- 4)、kernel power 模块经过 resume systemcore ->恢复全局 cpu 中断 ->拉起 nonboot cpus ->唤醒 dev(resume_noirq ->resume_early)
- 5)、mcu_mgr 模块在 resume_noirq 层被唤醒后判定当前是 SOC 自唤醒, atf 拉 gpio 让 MCU 唤醒, 阻塞并等待 MCU 的唤醒结果
- 6)、MCU 唤醒完成后拉 gpio 通知 SOC 自己唤醒完成。SOC 的 mcu_mgr 模块在收到 MCU 的唤醒结果后: 成功则继续唤醒;
- 7)、kernel power 模块检测到 resume_noirq 成功执行完成, 则继续唤醒 dev(resume_early→resume ->complete)→取消冻结, 然后唤醒 sleepmanager 调用的休眠函数。
- 8)、sleepmanager 模块收到调用休眠返回后, 判定唤醒成功且唤醒源为 soc rtc。
- 9)、sleepmanager 模块通过写虚拟机节点唤醒 TBOX。
- 10)、TBOX 系统的 kernel power 经过 resume systemcore ->恢复全局 cpu 中断 ->拉起 nonboot cpus ->唤醒 dev(resume_noirq →resume_early→resume ->complete)→取消冻结。然后返回 power hal 的调用函数。
- 11)、TBOX 系统的 power hal 模块收到返回唤醒后同步反馈给 sleepmanager(tbox)。
- 11.1)、sleepmanager(tbox)收到唤醒后, 判定唤醒成功且唤醒源为 SOC rtc 和闹钟事件后, 根据识别到的具体闹钟事件执行对应业务。通知 power hal 当前的子唤醒原因。
- 11.1.1)、power hal 模块通知子唤醒原因给 SOS 系统的 sleepmanager 模块。
- 11.1.2)、sleepmanager 模块根据子因通知 MCU 系统 PowerMode 模块当前状态。MCU 系统 PowerMode 模块开启强制保活的倒计时。
- 11.2)、sleepmanager(tbox)唤醒中间件各业务服务
- 12)、TBOX 系统的 power hal 模块唤醒 sleepmanager(tbox)后发送“开始唤醒消息”给 sleepmanager。

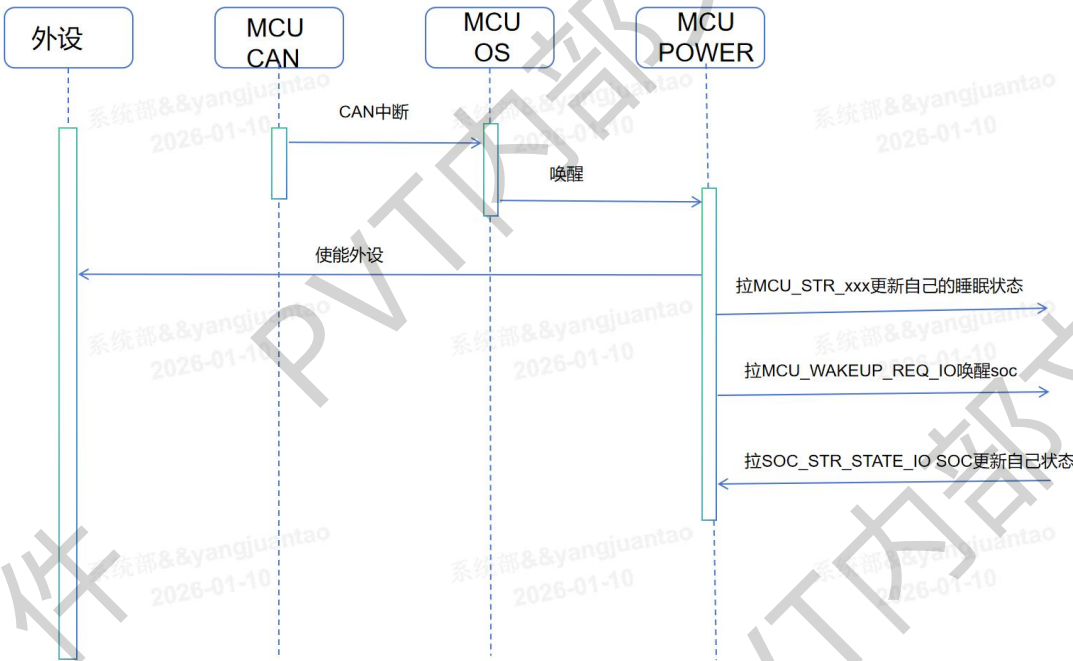
2. MCU 扩展

2.1 MCU/SOC 沟通 gpio 设定

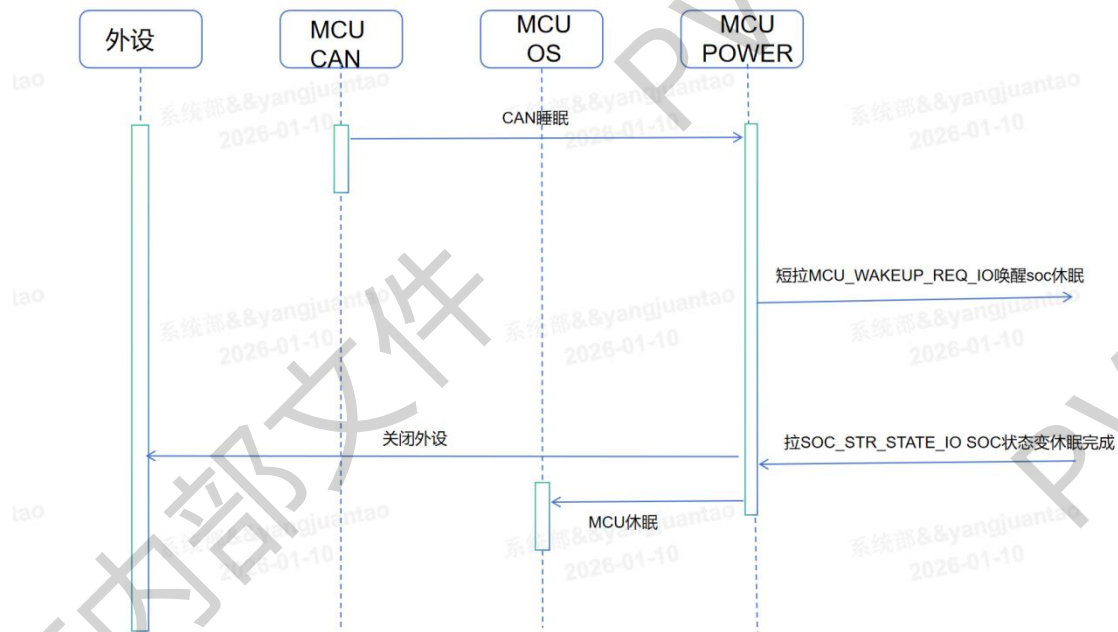
本文使用名称	使用场景	写	读	解释
MCU_WEAKUP_REQ_IO	MCU唤醒SOC场景	MCU-PowerMode	SOC-MTK	用于MTK定义MCU来拉起SOC的gpio
SOC_STR_STATE_IO		SOC-MTK	MCU-PowerMode	用于MTK定义的MCU拉起SOC后，SOC回复自己(睡眠/唤醒)状态给MCU的gpio
SOC_WEAKUP_ xxx	SOC唤醒MCU场景	SOC-atf	MCU-IRQ	用于定义的SOC来拉起MCU的gpio
MCU_STR_ xxx		MCU-PowerMode	SOC-kernel	用于定义的SOC来拉起MCU后，MCU回复自己(睡眠/唤醒)状态给SOC的gpio

2.2 使用场景介绍

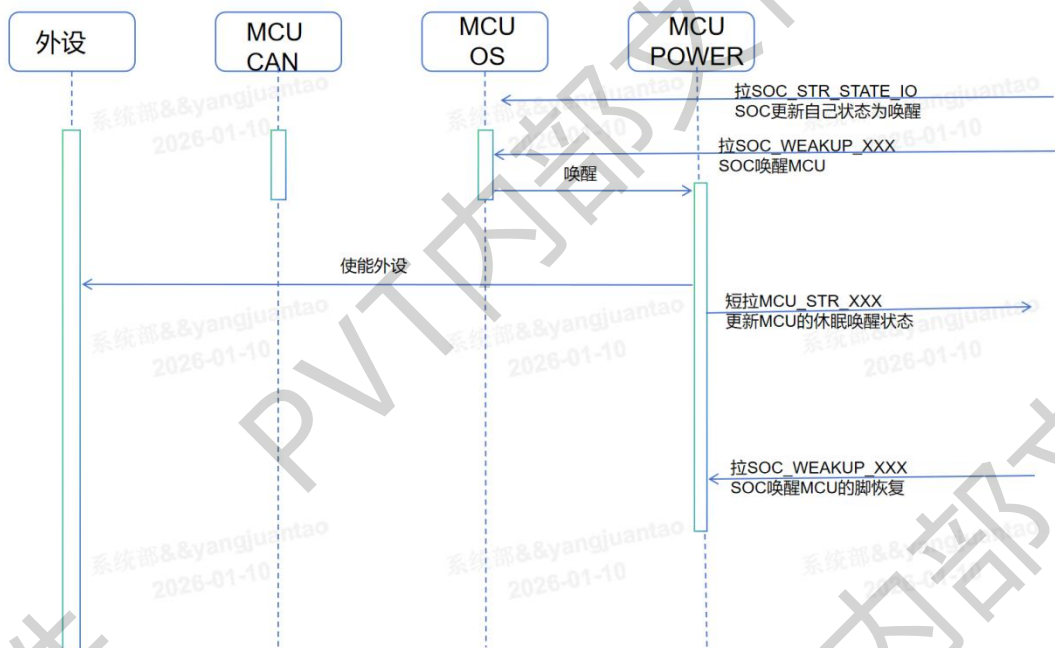
2.2.1 CAN 唤醒



2.2.2 休眠



2.2.3 SOC 唤醒 MCU



2.3 MCU 保活机制

在一些模式和状态下（如长连接等），按照 can 信号条件，是会触发 STR 的，但需求要求唤醒后需要保活 60 秒，等待系统执行一些事情，所以设计了因这类唤醒，做 MCU 的保活

唤醒因子	是否需要MCU基础短暂保活	是否需要不断刷新保活时间	保活退出条件	保活时间更新
can网络恢复	否	否		
IGN硬线	否	否		
7天计时进深睡眠	是	否		
ecall按键	是	是，不断重复		
短信唤醒	是	否	保活超时	收到TBOX经SOS上MIS发来的CAN网络保持的消息，根据参数重置为对应的保活时间
数字远控	是	否	保活超时	收到TBOX经SOS上MIS发来的CAN网络保持的消息，根据参数重置为对应的保活时间
ECALL回拨	是	是，不断重复	收到Sleepmanager发来的ECALL业务结束通知	暂无
长连接	是	否	收到sleepmanger发来的长连接完成通知	暂无
预约升级	是	是	保活超时或者进入远程模式	收到TBOX经SOS上MIS发来的CAN网络保持的消息，根据参数重置为对应的保活时间
预约充电	是	是	保活超时或者进入远程模式	收到TBOX经SOS上MIS发来的CAN网络保持的消息，根据参数重置为对应的保活时间
智能闹钟	是	否	保活超时	暂无
自动闭锁	是	否	保活超时	收到TBOX经SOS上MIS发来的CAN网络保持的消息，根据参数重置为对应的保活时间
异常唤醒（如modem校验失败）	是	否	收到因子，直接触发HUT睡眠	无